

NOAA Technical Memorandum NMFS-XXX-##

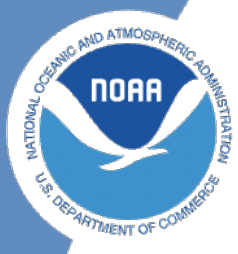
My GrandOrgue Implementation

Matt Grossi

January 2023

U.S. DEPARTMENT OF COMMERCE

National Oceanic and Atmospheric Administration
National Marine Fisheries Service
Northwest Fisheries Science Center



NOAA FISHERIES

My GrandOrgue Implementation

Making sense of scattered documentation and user experiences

Matt Grossi

Table of contents

Executive Summary **1**

1. Quick Start **2**
1.1. Hardware 2
1.2. Software 2
1.3. Troubleshooting 3

2. Hardware Requirements **4**
2.1. BLUF 4
2.2. Discussion 4

I. Setup **7**

3. MIDI Keyboard **8**

4. Digital Organ **10**

5. Software **12**
5.1. Raspberry Pi setup 12
5.2. GrandOrgue 16

6. Customization **21**
6.1. Bureå Funeral Chapel 21

References **23**

List of Figures

List of Tables

1.1. Virtual pipe organ on Raspberry Pi hardware setup 2

2.1. Total cost of my initial proof-of-concept setup. 5

3.1. Using a virtual pipe organ on Raspberry Pi with a MIDI-capable keyboard 8

4.1. Using a virtual pipe organ on Raspberry Pi with a MIDI-equipped digital organ 11

Executive Summary

This site documents my experience setting up, configuring, and using a GrandOrgue virtual pipe organ (VPO). As of 2026, there was little compiled documentation other than how to download the software. Information such as system and hardware requirements was scattered throughout the internet, but one generally had to find it through a search engine. Some of this information was later discovered within the GrandOrgue Help pages, but that was after downloading, installing, and digging into the software. I wanted to know what I needed *before* starting the project to determine whether it was feasible. For example, could I use a Raspberry Pi or would I need something more powerful? Lots of community user experiences were captured in hundreds of issues and thousands of discussions on the official GitHub repo. It all felt very scattered. This site attempts to organize my research, trials, and errors.

These pages are my contribution to the open source GrandOrgue project and its outstanding user community. I hope these pages help you build and enjoy your own GrandOrgue virtual pipe organ.

1. Quick Start

After everything has been set up the first time

1.1. Hardware

Table 1.1.: Virtual pipe organ on Raspberry Pi hardware setup

Connect...		...to...	...using
Monitor	□	Raspberry Pi	HDMI
Monitor	□	Raspberry Pi	USB, white port
Audio interface	□	Raspberry Pi	USB, blue port
Audio interface	□	MIDI keyboard	MIDI cable[1]
Computer keyboard / mouse	□	Raspberry Pi	USB, white first, then blue
Raspberry Pi	□	AC power	
MIDI Keyboard	□	AC power	
Headphones or speakers	□	audio interface (preferred)	Wired (preferred)

1.2. Software

1. Boot Raspberry Pi
2. Turn on keyboard
3. Launch GrandOrgue

[1] Be sure to connect **MIDI OUT** from the keyboard or organ to **MIDI IN** on the audio interface.

1.3. Troubleshooting

1. Verify the correct audio output is selected
2. If using an audio interface, ensure the monitor switches and volume knobs are set properly

2. Hardware Requirements

2.1. BLUF

The minimum requirements needed to get started with GrandOrgue are:

- Computer with at least 8 GB RAM (16+ GB preferred)
- Touch screen monitor (for interacting with virtual console controls)
- MIDI-USB cable or MIDI cable with a USB audio interface device for connecting the computer to the MIDI instrument
- MIDI-ready organ or keyboard

And, if you don't want the sound to play from your computer speakers:

- Audio cable to port audio from computer to an organ amplifier
  or

- Quality external speakers or headphones to connect directly to the computer

2.2. Discussion

GrandOrgue is free open source computer software supported for Windows, MacOS, and at least some Linux distributions. System and hardware requirements are summarized here, with the most important requirement being **RAM**.

My initial installation was intended to be a proof-of-concept learning experiment to assess whether or not it was something I wanted to delve further into and scale up. I already had an old (circa 1997) Casio MIDI keyboard[1], so I decided to use that as the workbench test case for experimentation. Then, once I knew what I was doing and felt comfortable with the technical components, I wanted to transfer the setup to a MIDI-ready digital church organ to try it out "in the field" with an actual organ console, decent speakers, and a room where an organ is regularly used.

[1] Be sure to connect **MIDI OUT** from the keyboard or organ to **MIDI IN** on the audio interface.

With this in mind, I sought portability with minimal spatial footprint. Desktop computers took up too much space and are not easily portable, so they were out. Without having a personal organ at home, I needed to interface with the software directly through the computer rather than via an organ console, so I wanted to have a touch screen monitor that could be positioned in close proximity to the music keyboard. While standard laptops with touch screens are portable and could work, I would have to always reach over the laptop keyboard to reach the screen, which was not ideal. My solution was to use a **Raspberry Pi** together with a simple **touch screen monitor**.

I opted for the Raspberry Pi 5 because it was the first and, as of 2026, only unit to contain 16 GB of RAM. I purchased a full CanaKit starter kit which included:

- Raspberry Pi 5 with 2.4Ghz 64-bit quad-core CPU and 64 GB of RAM
- 128GB Micro SD Card pre-loaded with 64-bit Raspberry Pi OS, USB MicroSD Card Reader
- Plastic protective case
- Low noise bearing system fan
- Heat sink
- 45W PD Power Supply
- 2 6-foot display cables

I bought a simple and compact 15.6” portable touch screen monitor for my primary means of interacting with GrandOrgue. An upgrade might be in store down the road, but this will due to start.

Unfortunately, the Raspberry Pi 5 does not have a dedicated 3.5mm headphone audio jack like its predecessor. Audio is outputted through HDMI or USB instead. I ended up buying a Behringer UMC204HD **USB audio interface** which would allow me to use a single USB port for both audio and to connect standard MIDI cables instead of a MIDI-USB cable. I also bought a 9.8-foot dual 5-pin **MIDI cable** to go along with that because I did not have one already.

Altogether, my expenses for the initial set up came out to be just over $600:

Table 2.1.: Total cost of my initial proof-of-concept setup.

Expense	Cost (USD)
Raspberry Pi 5 kit	$385
Behringer UMC204HD	$116
Touch screen monitor	$95
MIDI cable	$11
Keyboard	$0

2. Hardware Requirements

Expense	Cost (USD)
TOTAL	**$607**

Not bad for an authentic-sounding practice organ! The pages that follow describe how I set up, configured, and tested everything.

Part I.

Setup

3. MIDI Keyboard

Using GrandOrgue with a MIDI-capable keyboard

This page describes how to set up GrandOrgue with a single MIDI-capable keyboard.

1. Connect the touchscreen monitor to the Raspberry Pi (RPi) using an HDMI cable and a USB cable plugged into a white USB 2.0 port. The monitor needs both cables to handle video, audio, and mouse commands.
2. Connect the Behringer UMC204HD USB audio interface to the RPi using a USB cable plugged into a blue USB 3.0 port.
3. Connect the keyboard **MIDI OUT** to the audio interface **MIDI IN**. This enables the keyboard to control the GrandOrgue software. Since the software won't be controlling the keyboard in any way, there is no need for a second MIDI cable.
4. If using an external keyboard or mouse, connect to it to the RPi using either the white USB 2.0 port (preferred) or the remaining blue USB 3.0 port.
5. Connect the power supplies to the RPi and keyboard.

3.0.1. To summarize

Table 3.1.: Using a virtual pipe organ on Raspberry Pi with a MIDI-capable keyboard

Connect...		...to...	...using
Monitor	□	Raspberry Pi	HDMI
Monitor	□	Raspberry Pi	USB, white port
Audio interface	□	Raspberry Pi	USB, blue port
Audio interface	□	MIDI keyboard	MIDI cable[1]
Computer keyboard / mouse	□	Raspberry Pi	USB, white first, then blue
Raspberry Pi	□	AC power	

[1] Be sure to connect **MIDI OUT** from the keyboard or organ to **MIDI IN** on the audio interface.

3. MIDI Keyboard

Connect…		…to…	…using
MIDI Keyboard	→	AC power	
Headphones or speakers	→	audio interface (preferred)	Wired (preferred)

4. Digital Organ

Using GrandOrgue with a MIDI-equipped organ

This page describes how to set up GrandOrgue with a MIDI-equipped digital organ.

1. Connect the touchscreen monitor to the Raspberry Pi (RPi) using an HDMI cable and a USB cable plugged into a white USB 2.0 port. The monitor needs both cables to handle video, audio, and mouse commands.
2. Connect the Behringer UMC204HD USB audio interface to the RPi using a USB cable plugged into a blue USB 3.0 port.
3. Connect the organ **MIDI OUT** to the audio interface **MIDI IN**. This enables the organ console to control the GrandOrgue software. Since the software won't be controlling the organ console in any way, there is no need for a second MIDI cable. If the organ has separate switched and unswitched MIDI ports, using switched ports will require the divisional MIDI stops to be engaged in order for the virtual organ to sound through the speakers, while using unswitched ports will allow the virtual organ to speak regardless of whether the divisional MIDI stops are engaged. The second option is more intuitive for most applications.
4. To utilize the digital organ speakers, connect the audio interface **AUDIO OUT** to audio line inputs on the organ. This will require either two 1/4" TRS or a two-channel RCA cable, depending on the organ.
5. If using an external keyboard or mouse, connect to it to the RPi using either the white USB 2.0 port (preferred) or the remaining blue USB 3.0 port.
6. Connect the power supplies to the RPi.

4.0.1. To summarize

Table 4.1.: Using a virtual pipe organ on Raspberry Pi with a MIDI-equipped digital organ

Connect…		…to…	…using
Monitor	→	Raspberry Pi	HDMI
Monitor	→	Raspberry Pi	USB, white port
Audio interface	→	Raspberry Pi	USB, blue port
Audio interface **MIDI IN**	→	Organ **MIDI OUT** (unswitched preferred)	MIDI cable
Audio interface **AUDIO OUT**	→	Organ audio **LINE IN**	Two-channel RCA cable or two 1/4” TRS cables
Computer keyboard / mouse	→	Raspberry Pi	USB, white first, then blue
Raspberry Pi	→	AC power	
MIDI Keyboard	→	AC power	

5. Software

Setting up the GrandOrgue software

5.1. Raspberry Pi setup

Raspberry Pi (RPi) devices run Debian-based Raspberry Pi OS. GrandOrgue needs the 64-bit Raspberry Pi OS in order to utilize all of the installed RAM (a 32-bit OS would cap the usable RAM at 4 GB, regardless of what is installed in the system.) The 64-bit OS conveniently comes pre-installed on the CanaKit RPi.

Before installing anything, download and install any OS updates:

```
sudo apt update && sudo apt full-upgrade -y
sudo reboot
```

GrandOrgue recommends X11 to properly handle window sizing and positioning. To switch the display mode:

1. Open a terminal and run
   ```
   sudo raspi-config
   ```
2. Use the down arrow key to select **Advanced Options**
3. In the next menu, select **Wayland**
4. Select **X11**, then **<OK>**
5. Use the right arrow key to select **<Finish>** and then **Reboot**

The touch monitor is plug-and-play. Since the RPi 5 only has a single USB-C port, which is used for the power supply, the monitor needs to be connected to the RPi via both HDMI for data and USB for power. I used the HDMI to mini-HDMI cable that came with the RPi and the USB-C to USB-A cable that came with the monitor. The RPi needed to be rebooted after everything was connected for the touch screen to work properly.

5.1.1. Set Screen Resolution

The Pi occasionally fails to read the monitor's EDID (Extended Display Identification Data) on boot and defaults to 1024x768. Force the correct resolution by adding these lines to the boot config:

```
sudo nano /boot/firmware/config.txt
```

Add at the bottom:

```
hdmi_force_hotplug=1
hdmi_group=2
hdmi_mode=82
hdmi_force_hotplug=1 - forces Pi to treat HDMI as always connected
hdmi_group=2 - sets DMT (Display Monitor Timings) mode for computer monitors
hdmi_mode=82 - forces 1920x1080 at 60Hz
```

5.1.2. Touch screen right-click

There was no discernable way to right-click on the touch screen by default on the X11 release of Raspberry Pi OS. This can be fixed by telling X11, the software layer that manages the display and interprets input, to treat a specific pattern of touch behavior, such as briefly holding a finger still on a spot, as a right-click signal. To do this, we need to set three settings:

1. `EmulateThirdButton` to turns the feature on
2. `EmulateThirdButtonTimeout` to set how long to hold before it triggers, in milliseconds. 750 is long enough that normal taps don't accidentally trigger it, but short enough to feel responsive.
3. `EmulateThirdButtonMoveThreshold` to set how many pixels your finger is allowed to drift during the hold before the gesture is cancelled. This prevents accidental right-clicks when you're just resting your finger briefly while scrolling or moving around the screen.

Open a terminal window and enter:

```
sudo nano /usr/share/X11/xorg.conf.d/99-ilitek-touch.conf
```

If this file exists, add the following inside the `Section "InputClass"` block:

```
Option "EmulateThirdButton" "1"
Option "EmulateThirdButtonTimeout" "750"
Option "EmulateThirdButtonMoveThreshold" "30"
```

If the file that opens is blank, paste the following in it:

```
Section "InputClass"
    Identifier "ILITEK Touch"
    MatchProduct "ILITEK ILITEK-TP"
    MatchIsTouchscreen "on"
    Option "TransformationMatrix" "1 0 0 0 1 0 0 0 1"
    Option "EmulateThirdButton" "1"
    Option "EmulateThirdButtonTimeout" "750"
    Option "EmulateThirdButtonMoveThreshold" "30"
EndSection
```

This does two things: maps the touch device correctly to the display and enables long-press (750ms hold) as right-click emulation. Save the file and reboot the machine. Touch screen right-clicking should now be supported.

5.1.3. On screen keyboard

The X11 release of Raspberry Pi OS also does not natively support an on-screen touch keyboard. To avoid needing an external keyboard, install and configure **onboard**, a decent on-screen keyboard for X11 on Raspberry Pi:

```
sudo apt install onboard
```

To make this keyboard automatically appear when needed, like on a tablet or smartphone, also install **AT-SPI2**, the Linux Assistive Technology Service Provider Interface, and enable accessibility:

```
sudo apt install at-spi2-core
gsettings set org.gnome.desktop.interface toolkit-accessibility true
```

Configure the keyboard to your liking using its built-in settings:

```
onboard-settings
```

I chose to dock it to the bottom of the screen and stretch across the screen, like on a tablet:

- Window enable “Dock to screen edge”
- Window Docking Settings select “Shrink workarea”, “Expand on landscape screens”, and “Expand on portrait screens”
- Window Docking Settings select “Dock to screen edge: Bottom”

And to auto-show when needed:

- Auto-show “Auto-show when editing text”

This does not behave as flawlessly or as conveniently as one is accustomed to on mobile devices, but it is close enough for a system that will primarily be used as a virtual organ. For example, one caveat is that the keyboard application has to be running in order for auto-show to work. Also, the keyboard itself is known to work fine in some applications, not at all in some, and be hit-or-miss in others. To handle this, add a toggle button to the taskbar that can be used to launch the keyboard when first needed or to manually show it if it does not automatically show in some application. This requires three steps:

1. Create a system-wide `.desktop` file:

   ```
   sudo nano /usr/share/applications/onboard.desktop
   ```

   In it, add the following:

   ```
   [Desktop Entry]
   Name=On-Screen Keyboard
   Comment=Toggle On-Screen Keyboard
   Exec=bash -c 'DISPLAY=:0 killall onboard || DISPLAY=:0 onboard'
   Icon=onboard
   Type=Application
   Categories=Utility;
   ```

2. Modify the system `panel` file to add it to the taskbar:

```
nano ~/.config/lxpanel/LXDE-pi/panels/panel
```

   Add the following block to the end of the file:

```
Plugin {
  type=launchtaskbar
  Config {
    Button {
      id=onboard.desktop
    }
  }
expand=0
}
```

3. Open the keyboard application on startup:

```
nano ~/.config/lxsession/LXDE-pi/autostart
```

   Add to the end of the file:

```
@onboard
```

After these steps, reboot the machine for the settings to take affect.

5.2. GrandOrgue

5.2.1. Installation

From the GrandOrgue releases GitHub, download the `arm64.deb` architecture package. As of May 2026, the current release was `grandorgue_3.17.1-1_arm64.deb`. It can also be downloaded directly in the terminal:

```
wget https://github.com/GrandOrgue/grandorgue/releases/download/3.17.1-1/grandorgue_3.17.1-
```

> **Note**
>
> `grandorgue` requires **wxWidgets 3.2** and `grandorgue-wx30` requires **wxWidgets 3.0**. Raspberry Pi OS Bookworm ships with wxWidgets 3.2, so the standard `grandorgue` package above is the correct one.

Install the software using either the downloaded installer or terminal:

```
sudo apt install ./grandorgue_3.17.1-1_arm64.deb
sudo apt --fix-broken install
```

You might get a `Permission Denied` warning, but it can be ignored:

```
N: Download is performed unsandboxed as root as file '/home/username/grandorgue_3.17.1-1_ar
```

It is not a fatal error; the application should have installed fine.

Verify the installation by looking for it in the **Sound & Video** application menu. Launch the program. It should open and load the demo organ.

5.2.2. Configuration

Next, we need to map MIDI channels between GrandOrgue and the instrument. Open GrandOrgue and go to **File Settings**

1. **MIDI Devices** tab: Select UNC204HD as both the **MIDI input device** and **MIDI output device**.
2. **Audio** tab:
    - Set **Sample Rate** to 48000 and **Samples per buffer** to 128
    - Under **Sound ports**, select **RtAudio** and **RtAudio alsa**
    - In the **Mapping output** panel, under **Audio Output**, set the Device to **RtAudio: alsa: Default ALSA Device [50 ms requested]** to route the audio through the built in monitor speakers for now. (To set this, click the name of the current device, then click **Change**.) See Audio for instructions on changing this later.

To reduce audio lag:

1. **File Settings Audio**
2. Set the **Samples per buffer** to 256 or 128 (defaults to 512).

The lower the buffer size, the lower the latency. At 48,000 Hz sample rate, 128 samples per buffer supposedly gives about 2.7 milliseconds of latency, which is completely imperceptible. The tradeoff is that very low buffer sizes can cause audio crackle or dropouts if the RPi can't process audio fast enough. The Pi 5 should, however, be plenty powerful enough to handle 128 samples without trouble, especially with the demo sample set.

Even at 128 samples/buffer, I still noticed a lag, but it was better than when I started. It is possible that some delay is "built in" as the result of the sample recording. This remains to be determined. I am accustomed to old-technology, non-realistic digital organs on which the sound is instantaneous, whereas many pipe organs have a slight (fraction of a second) lag between the time a key is depressed and the sound is actually heard.

The overall volume can be controlled via the GrandOrgue menu bar using the **+** and **-** buttons next to the small speaker icon. Raise the volume cautiously, watching the audio bars as you play. Going to loud will cause the audio to clip (max out), leading to distortion.

5.2.3. Updating

As of late 2026, GrandOrgue does not include a built-in software update method. While it does display an alert in the menu bar when an update is available, one must manually download and install it. The process is the same as the initial installation:

```
wget https://github.com/GrandOrgue/grandorgue/releases/download/X.X.X-X/grandorgue_X.X.X-X_
sudo apt install ./grandorgue_X.X.X-X_arm64.deb
sudo apt --fix-broken install
```

where `X.X.X-X` is the version release to download. Since I do not use Linux enough to remember these commands, nor can I ever remember the correct asset, I created a simple Shell script to handle it for me:

For convenience, and until I remember that it exists, I put this script on my Desktop alongside GrandOrgue. To use it, open a terminal window, navigate to the Desktop, and launch the code, passing the desired release version as a command line argument:

```
./update_grandorgue.sh 3.17.3-1
```

5.2.4. Audio

Several audio routing options exist, but GrandOrgue needs to be configured properly for each one to work. High-quality headphones or external speakers are highly recommended.

To use the touchscreen built-in speakers or headphones or external speakers plugged into the touchscreen headphone jack:

- In the **Mapping output** panel, under **Audio Output**, set the Device to **RtAudio: alsa: Default ALSA Device [50 ms requested]**. (To set this, click the name of the current device, then click **Change**.)

To use headphones or external speakers connected to the USB audio interface (preferred method):

- In the **Mapping output** panel, under **Audio Output**, set the Device to **RtAudio: alsa: UMC204HD 192k (USB Audio) [50 ms requested]**.
- Turn the **MIX** knob fully clockwise to PS (playback signal) to hear only the audio playback from GrandOrgue. (If set counterclockwise to IN, you won't hear anything.)
- Adjust the volume using the **PHONES** volume knob.
- Toggle the **MONITOR** button until you hear sound. In my case, it has to be out.

Use the first configuration (**RtAudio: alsa: Default ALSA Device [50 ms requested]**) for Bluetooth headphone as well, since the audio will be handled by the operating system rather than GrandOrgue. My experience, however, has been that bluetooth headphones experience a fairly significant lag in audio, so I use wired headphones.

Since my old keyboard did not have an Audio In jack, I could not channel the sound back to the keyboard speakers – which was just as well, because they are not very good anyway. The best (but not perfect) option was wired headphones plugged into the UNC204HD interface. The only headphones I owned had a 1/8" plug, so I had to use a 1/4"-1/8" adapter to connect them. There was a slight buzz, possibly resulting from the extra connection. It was annoying, but not bad enough to abandon the setup.

Listing 5.1 `update_grandorgue.sh`

```
#!/bin/bash
# update-grandorgue.sh
# Downloads and installs a GrandOrgue release from GitHub
# Usage: ./update-grandorgue.sh <version>
# Example: ./update-grandorgue.sh 3.17.3-1

set -e  # Exit immediately if any command fails

# --- Check argument ---
if [ -z "$1" ]; then
    echo "Usage: $0 <version>"
    echo "Example: $0 3.17.3-1"
    echo ""
    echo "Find the latest version at:"
    echo "https://github.com/GrandOrgue/grandorgue/releases"
    exit 1
fi

VERSION="$1"
ARCH="arm64"
FILENAME="grandorgue_${VERSION}_${ARCH}.deb"
URL="https://github.com/GrandOrgue/grandorgue/releases/download/${VERSION}/${FILENAME}"
DOWNLOAD_DIR="/tmp"
DOWNLOAD_PATH="${DOWNLOAD_DIR}/${FILENAME}"

echo "============================================"
echo " GrandOrgue Updater"
echo "============================================"
echo " Version : $VERSION"
echo " File    : $FILENAME"
echo " URL     : $URL"
echo "============================================"
echo ""

# --- Download ---
echo "Downloading $FILENAME..."
wget -O "$DOWNLOAD_PATH" "$URL"

if [ ! -f "$DOWNLOAD_PATH" ]; then
    echo "Error: Download failed. Check the version number and try again."
    exit 1
fi

echo "Download complete."
echo ""

# --- Install ---
echo "Installing $FILENAME..."
sudo apt install -y "$DOWNLOAD_PATH"
```

6. Customization

Using different organ sample sets

The default sample set is great to get started with, but it's rather small. The beauty of virtual pipe organ (VPO) software is the ability and flexibility to use any number of sample sets. Many sample sets of various sizes exist, from small single-division organs to large many-division instruments. One constraint is disk storage: one needs to have enough space to download and store the sample set. The primary constraint with choosing samples to use, however, is RAM: the sample set must fit in the computer's RAM to work properly. Thus, my constraint was 16 GB RAM that came on the Raspberry Pi 5. What this means in practice:

- **Under 8 GB:** loads comfortably, plenty of headroom
- **8–14 GB:** should work fine, some breathing room for OS
- **14-15 GB:** getting risky; the OS and GrandOrgue itself need some RAM too, so you don't want to use all 16 GB for the sample set
- **Over 15 GB:** will not work; not enough memory for GrandOrgue and the OS

Since I only had one keyboard with no pedalboard, it made sense to consider single-manual samples. I could use larger ones, but I could only play one manual. The workaround would be mapping the Great division to my keyboard and relying on couplers to play other manuals from the Great manual. (I might also be able to map all manuals to the same keyboard and essentially multiply the number of stops available, but I haven't tested this yet.)

6.1. Bureå Funeral Chapel

The first organ I tried was the Bureå Funeral Chapel from Lars Virtual Pipe Organs. This is a small chamber organ with one manual, pedals, and nine stops. All of the manual stops are split stops with separate drawknobs for bass (first two octaves) and everything else. Split stops provide greater flexibility on a small instrument, allowing

the organist to use different combinations of stops on one manual instead of across two manuals. Since I only have one keyboard, this seemed like a good starting point.

First, download the `.rar` file. I needed to install the “unrar” utility the first time in order to extract this:

```
sudo apt install unrar
```

Next, unpacked it into the `GrandOrgue/Organs/` directory:

```
unrar x Burea_Funeral_Chapel.rar /home/username/GrandOrgue/Organs/Burea_Funeral_Chapel/
```

Finally, open GrandOrgue, go to **File** → **Open**, find and select the desired organ sample, and click **Open**. The new organ should load and be ready for use.

References